

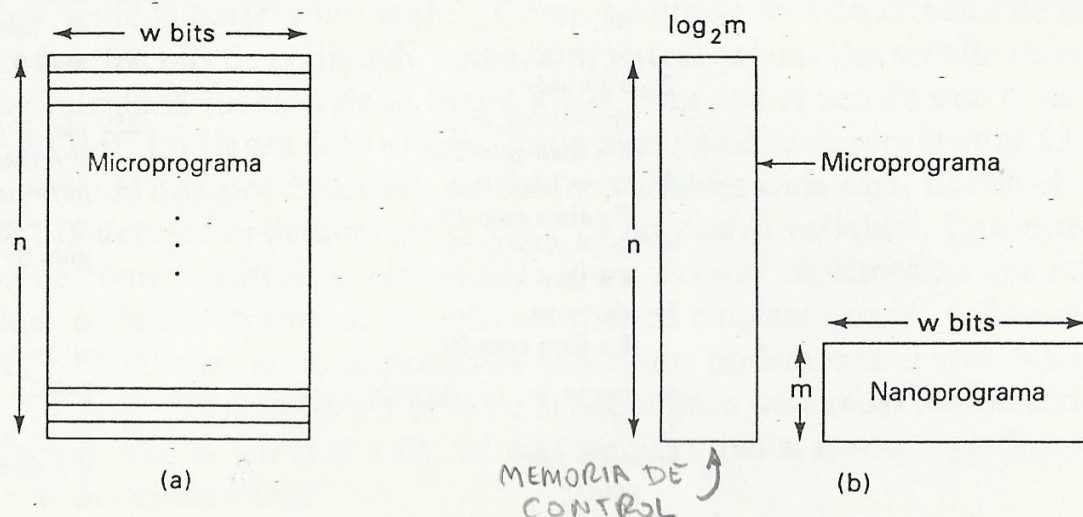


Fig. 4-21. (a) Microprograma convencional. (b) Nanoprograma correspondiente si sólo aparecen en el microprograma m microinstrucciones.

El microprograma se ejecuta como sigue. Primero se extrae una palabra de la memoria de control y se usa para seleccionar una palabra de la nanomemoria, que se extrae y se guarda en el registro de microinstrucción. Los bits de este registro se utilizan como señales de control durante un ciclo. Al finalizar este ciclo, se extrae la siguiente palabra de la memoria de control y el proceso se repite.

Por ejemplo, supongamos que el microprograma original tiene 4096×100 bits pero sólo aparecen 128 microinstrucciones distintas. Bastaría una nanomemoria de 128×100 bits para guardar las microinstrucciones necesarias. La memoria de control tendría ahora 4096×7 bits, apuntando cada una de sus palabras a una nanoinstrucción. El ahorro de memoria en este ejemplo sería:

$$\text{ahorro} = 4096 \times 100 - 4096 \times 7 - 128 \times 100 = 368\,128 \text{ bits}$$

El precio que debe pagarse por el ahorro de memoria es una ejecución más lenta. La máquina con la memoria de control de dos niveles funcionará con menor rapidez que la original, debido a que el ciclo de extracción requiere ahora dos referencias a memoria, una a la memoria de control y otra a la nanomemoria. Estas dos extracciones no pueden traslaparse.

La nanoprogramación es verdaderamente eficaz cuando se usan mucho las mismas microinstrucciones. Si se pudieran contar como iguales dos microinstrucciones casi iguales, el microprograma contendría menos microinstrucciones distintas, pero más frecuentemente usadas. Se puede variar la idea básica para permitir esto con sólo utilizar como parámetro las palabras de la nanomemoria. Por ejemplo, dos microinstrucciones pueden diferir solamente en un campo que diga qué registro ha de sacarse a cierto bus. Poniendo el número del re-

gistro en la memoria de control, en lugar de ponerlo en la nanomemoria (es decir, omitiendo el campo del registro de las nanoinstrucciones), las dos microinstrucciones podrían apuntar a la misma nanopalabra. Cuando ésta se extraiga y se ponga en el registro de microinstrucción, el campo de registro se tomará de la micromemoria y no de la nanomemoria.

Naturalmente, con este mecanismo aumenta la anchura de la memoria de control, que puede o no compensar la ganancia obtenida con una nanomemoria más pequeña.

4.5.3. Mejora del rendimiento

Aunque el objetivo de la nanoprogramación es reducir el tamaño de la memoria de control, incluso al precio de una ejecución más lenta, también se ha invertido gran cantidad de tiempo y esfuerzo en el objetivo opuesto: intentar acelerar la ejecución, incluso al precio de una mayor memoria de control. Estos dos criterios de diseño aparentemente incompatibles tienen que ver con dos estrategias de mercadotecnia: la de máquinas baratas y la de máquinas rápidas. En esta sección examinaremos algunas de las maneras en que se puede modificar nuestra sencilla microarquitectura para que las máquinas funcionen más rápido.

Hasta ahora hemos supuesto que los cuatro subciclos tenían la misma duración. Aunque esto es sencillo, pocas veces produce el mejor rendimiento, ya que invariablemente uno de los cuatro necesita mucho más tiempo que los otros tres. Si los cuatro duraran lo mismo que el peor de los casos, la velocidad de la máquina disminuiría notablemente. La forma de corregir esto es permitir que la duración de cada subciclo sea independiente de la de los demás. La duración de cada subciclo debe ajustarse a la necesaria para realizar su función.

Aunque esto constituye un paso hacia adelante en la dirección correcta, es aún demasiado conservador, ya que la duración de cada subciclo está todavía determinada por el peor de los casos en ese subciclo. Considérese, por ejemplo, el subciclo 3 de nuestros ejemplos. Si la función de la ALU es la suma, probablemente se necesite más tiempo (debido a los retrasos en la propagación del acarreo) que si la operación fuera simplemente seleccionar A. Si la duración del subciclo 3 se determinara en el momento del diseño, debería ser la de la suma y no la de seleccionar A. Otra estrategia consistiría en que cada operación que va a realizarse determine la duración del subciclo, de modo que sea tan corta como sea posible.

Una vez tomada la decisión de hacer que la duración de cada subciclo dependa de la operación, debemos encontrar una forma de llevar a la práctica esta estrategia. Acelerar o desacelerar el reloj es una técnica difícil, por eso lo que se hace es conseguir un reloj maestro con un periodo mucho más corto que la du-